Lista de Exercícios – Retropropagação (Backpropagation)

Prof. Chauã Queirolo

Nível 1 – O que é "treinar" e como um neurônio calcula

Exercício 1 – O que significa treinar uma rede?

Conceito. Uma rede neural é uma máquina com milhares de botões de ajuste, chamados **pesos**. Pense num rádio antigo: você gira o botão até a estação ficar nítida. **Treinar = girar os botões (pesos) até a saída ficar certa.** Os dados não mudam; quem muda são os botões.

Perguntas

1. Em uma frase, o que é um "peso" dentro de uma rede neural?
2. Durante o treinamento, o que é ajustado: os dados de entrada ou os pesos?
3. Verdadeiro ou falso: "quando a rede aprende, ela reescreve o próprio código-fonte". Justifique.

Exercício 2 – Como um neurônio faz sua conta (passo para frente)

Conceito. Um neurônio é como um gerente que recebe opiniões de vários funcionários (entradas). Cada opinião tem uma **importância** (peso). Ele soma tudo, adiciona um empurrãozinho fixo (o **bias**) e passa o resultado por um "porteiro" chamado **função de ativação**, que decide o que sai. Fazer essa conta da entrada até a saída é o **passo para frente (forward pass)**.

Perguntas. Um neurônio recebe x1 = 2 e x2 = 3, com pesos 0,5 e −1 e bias = 1.

1. Qual é a soma ponderada (x1 × peso1 + x2 × peso2 + bias)?
2. A ativação é a ReLU ("se for negativo, devolve 0; senão, devolve o próprio valor"). Qual é a saída do neurônio?
3. Em linguagem de gente: o que significa um peso **negativo**?
4. Em uma frase, o que é o "forward pass"?

Exercício 3 – O caminho da informação em várias camadas

Conceito. Uma rede multicamadas é uma **linha de montagem**: a camada de entrada recebe os dados, as camadas **ocultas** transformam, e a camada de saída entrega a resposta. Cada ligação entre dois neurônios carrega um peso, então **mais neurônios = mais botões para ajustar**.

Perguntas. Considere uma rede 3-4-2 (3 neurônios de entrada, 4 na camada oculta, 2 na saída).

1. Quantas camadas ela tem?
2. Quantos pesos existem entre a entrada e a oculta? E entre a oculta e a saída? Qual o total (ignore os bias)?
3. Por que a camada do meio se chama "oculta"?

4. Se dobrarmos a camada oculta para 8 neurônios, o total de pesos dobra, mais que dobra ou menos que dobra? Calcule.

Nível 2 – Erro e ativação: medindo o tamanho do estrago

Exercício 4 – Função de perda (a "nota da prova" da rede)

Conceito. Para corrigir um erro, antes é preciso **medir** o erro. A **função de perda** (*loss*) é um único número que diz o quão errada a rede está. Perda **0** = gabaritou. Perda **alta** = chutou tudo errado. Ela é o "quadro de notas" que guia o treino: sem ela, a rede não sabe nem se está melhorando.

Perguntas

1. Perda alta é boa ou ruim? E perda zero, o que indica?
2. Por que precisamos de **um número** em vez de só dizer "tá meio errado"?
3. Uma rede deve prever o preço de uma casa. O preço real é R$ 300 mil. A rede A previu R$ 310 mil e a rede B previu R$ 500 mil. Qual tem perda maior? Qual você preferiria?
4. Durante o treino, o que esperamos ver acontecer com a perda ao longo do tempo?

Exercício 5 – Por que a rede precisa de função de ativação?

Conceito. Imagine empilhar várias camadas que só "somam e multiplicam": o resultado final seria sempre uma conta simples e reta, como se a rede tivesse uma camada só. A **ativação** é a "dobra" que quebra essa reta e permite aprender coisas curvas e complicadas. Duas comuns: **Sigmóide** (espreme qualquer número para ficar entre 0 e 1, tipo "probabilidade") e **ReLU** (zera o que é negativo e deixa o positivo passar).

Perguntas

1. O que acontece, na prática, se removermos as ativações de uma rede com 10 camadas ocultas? Ela fica mais poderosa, igual a uma camada só ou pior que isso?
2. Qual ativação (Sigmóide ou ReLU) faz sentido na **saída** de uma rede que responde "qual a chance deste e-mail ser spam?" Por quê?
3. Complete: a ReLU transforma −5 em ___, 0 em ___ e 7 em ___.
4. Um perceptron simples (1 neurônio) separa dados com uma **linha reta**. Cite um tipo de problema em que uma reta não basta.

Exercício 6 – Nem todo erro pesa igual (confiança errada dói mais)

Conceito. Em classificação, boas funções de perda (como a *entropia cruzada*) funcionam como um professor severo: errar **com muita confiança** custa MUITO mais que errar na dúvida. É a diferença entre "acho que é gato, mas não tenho certeza" e "JURO DE PÉS JUNTOS que é gato".

Perguntas. A imagem é de um **cachorro**.

1. A rede X disse "99% gato". A rede Y disse "51% gato". Qual tem perda maior?
2. Por que é desejável que a perda puna mais a confiança errada?
3. Qual rede você acha que vai aprender mais rápido com essa correção? Justifique em uma frase.

Nível 3 – A ideia da retropropagação: quem teve culpa no erro?

Exercício 7 – Qual peso errou? (a ideia de "culpa")

Conceito. A rede errou. Existem milhares de pesos. **Qual deles gira para qual lado, e quanto?** Imagine um trabalho em grupo que tirou nota baixa: o professor quer saber quanto cada aluno contribuiu para o desastre. Esse "grau de culpa" de cada peso é o que, no jargão, chamamos de **gradiente**. Você não precisa calcular nada: basta entender que **gradiente = o quão responsável aquele peso é pelo erro, e em qual direção ele deve ser corrigido** (como uma bússola).

Perguntas

1. Em uma frase, o que o gradiente de um peso informa?
2. Um peso tem "culpa quase zero". O que isso significa: ele deve ser alterado muito ou quase nada?
3. Dois pesos: A com culpa alta e B com culpa baixa. Quem recebe o maior ajuste?
4. Por que não dá para simplesmente "chutar" novos valores para todos os pesos? (pense: milhões de botões).

Exercício 8 – A culpa viaja de trás para frente (o "nome" retropropagação)

Conceito. O erro aparece na **saída**, mas a causa está espalhada por todas as camadas anteriores. A retropropagação resolve isso como um **telefone sem fio ao contrário**: o erro é medido no fim e a culpa vai sendo repassada para a camada anterior, depois para a anterior, até chegar no começo. Cada camada recebe a culpa da camada seguinte, divide entre seus pesos e repassa o resto. Por isso o nome: **retro** (para trás) + **propagação** (espalhar). O nome técnico do truque matemático é *regra da cadeia*, mas aqui a gente só precisa do ingrediente: "reaproveitar a conta da camada da frente para não refazer tudo do zero".

Perguntas

1. Por que a culpa é calculada **de trás para frente** e não da entrada para a saída?
2. Compare com um pipeline de fábrica: o produto saiu com defeito no final. Como você descobriria qual etapa causou o problema? Como isso se parece com a retropropagação?
3. Por que reaproveitar contas das camadas seguintes torna o algoritmo **rápido**? (O que aconteceria se cada peso precisasse de uma investigação do zero?)
4. Verdadeiro ou falso: "a retropropagação é o algoritmo que faz a rede dar a resposta". Corrija se for falso.

Exercício 9 – O ciclo completo de um passo de treino

Conceito. Um passo de treino é uma receita de 4 etapas que se repete milhares de vezes: **(1)** passo para frente, **(2)** medir o erro, **(3)** retropropagar a culpa, **(4)** ajustar os pesos. Quem confunde a ordem confunde tudo.

Perguntas

1. Coloque na ordem correta: (a) ajustar os pesos · (b) calcular a perda · (c) retropropagar · (d) passo para frente.
2. Em qual das etapas os pesos efetivamente **mudam**? Em qual só se calcula a culpa?
3. Explique com suas palavras a diferença entre **retropropagação** (calcular as culpas) e **descida do gradiente** (usar as culpas para ajustar).

4. O que acontece se pularmos a etapa 4 e repetirmos 1, 2 e 3 mil vezes?

Nível 4 – Atualizando os pesos: o tamanho do passo importa

Exercício 10 – Taxa de aprendizado (o tamanho do passo)

Conceito. Imagine descer uma montanha no meio de um nevoeiro, sentindo o chão com o pé para saber onde é mais baixo. A culpa (gradiente) te diz **para onde** andar; a **taxa de aprendizado** diz **o tamanho do passo**. A regra do ajuste é só uma conta de padaria:

```
\text{peso novo} = \text{peso velho} - \text{taxa} \times \text{culpa}
```

Passo pequeno demais: você chega, mas leva uma eternidade. Passo grande demais: você pula o vale inteiro e cai do outro lado.

Perguntas. Um peso vale 2,0 e a culpa dele é +4. O valor ideal, que nós sabemos mas a rede não, está perto de 1,5.

1. Com taxa = 0,1, qual é o peso novo?
2. Com taxa = 1, qual é o peso novo? O que aconteceu em relação ao valor ideal?
3. Qual das duas taxas parece mais saudável para este passo? Por quê?
4. Descreva em uma frase o que acontece com a perda se a taxa for **pequena demais** e se for **grande demais**.

Exercício 11 – Batch, mini-batch, época e iteração

Conceito. Você não precisa corrigir os pesos só depois de ver **todos** os exemplos. Pense em estudar para prova:

- **Batch completo:** lê o livro inteiro e só depois ajusta seu método de estudo (estável, mas lento).
- **Estocástico (1 exemplo):** ajusta o método a cada pergunta (rápido, mas ruidoso e nervoso).
- **Mini-batch:** ajusta a cada capítulo (o meio-termo mais usado hoje).

Uma **época** é uma volta completa por todos os dados de treino. Uma **iteração** é um ajuste de pesos.

Perguntas. Um conjunto de treino tem 1.000 exemplos.

1. Com mini-batch de 100, quantas iterações acontecem em **uma época**?
2. Treinando por 20 épocas, quantos ajustes de pesos ocorrem no total?
3. Se usarmos batch = 1 (estocástico), quantas iterações por época?
4. Cite uma vantagem e uma desvantagem de usar o batch completo em vez de mini-batches.

Exercício 12 – A paisagem do erro (vales, buracos e planícies)

Conceito. Visualize a perda como um **relevo de montanhas**: a altura é o erro e cada posição é uma combinação de pesos. Treinar é caminhar morro abaixo. Mas o terreno tem **buracos pequenos** (mínimos locais, onde você acha que chegou ao fundo mas não chegou) e **planícies** (onde a culpa fica quase zero e você não sabe para onde ir).

Perguntas

1. A retropropagação garante achar o **melhor** conjunto de pesos possível? Justifique com a metáfora do nevoeiro.
2. Por que treinar a mesma rede duas vezes costuma dar resultados um pouco diferentes? (dica: onde você é "largado" na montanha no início?)
3. Por que o "ruído" dos mini-batches pode, às vezes, ajudar a sair de um buraco raso?
4. Em uma planície, a culpa é quase zero. Qual é o efeito sobre o tamanho do ajuste? Isso é bom ou ruim?

Nível 5 – Quando dá ruim: os vilões do treinamento

Exercício 13 – Gradiente que some (ou explode)

Conceito. Lembra do telefone sem fio? Na retropropagação, a culpa é **multiplicada** a cada camada que ela atravessa. Se cada camada multiplica por um número **pequeno** (a Sigmóide faz isso!), a mensagem vira um sussurro e, quando chega nas primeiras camadas, é silêncio: elas **não aprendem** (gradiente que desaparece, *vanishing*). Se multiplica por um número **grande**, a mensagem vira um grito ensurdecedor e os pesos dão saltos malucos (gradiente que explode). A ReLU ajuda porque deixa o valor positivo passar inteiro, sem espremer.

Perguntas. A culpa começa valendo 1 na saída e atravessa 5 camadas.

1. Se cada camada multiplica por 0,25, qual é a culpa ao chegar na primeira camada? (aproxime)
2. Se cada camada multiplica por 2, qual é a culpa ao chegar lá?
3. Qual dos dois casos faz as primeiras camadas "não aprenderem nada"? E qual deixa o treino instável?
4. Por que trocar Sigmóide por ReLU nas camadas ocultas costuma amenizar o problema do sussurro?

Exercício 14 – Por que não começar com todos os pesos iguais?

Conceito. Se todos os pesos começam com o mesmo valor (por exemplo, zero), todos os neurônios de uma camada fazem a mesma conta, recebem a mesma culpa e recebem o mesmo ajuste. São **clones**: dez alunos copiando a mesma cola. A rede perde a capacidade de ter neurônios especialistas. Por isso a inicialização usa **números aleatórios pequenos**, para quebrar a simetria.

Perguntas

1. O que acontece com neurônios de uma mesma camada se todos começam com pesos idênticos?
2. Por que "aleatório" resolve o problema?
3. Por que os números devem ser **pequenos** e não enormes? (dica: pense no Exercício 13)
4. Começar com todos os pesos em zero é um problema também para o **bias**, ou só para os pesos? Reflita e justifique.

Exercício 15 – Decorar x entender (overfitting e underfitting)

Conceito. Um aluno que **decora** as respostas do simulado tira 10 no simulado e 4 na prova de verdade. Isso é **overfitting**: a rede vai muito bem nos dados de treino e mal em dados novos. Já o aluno que nem abriu o livro vai mal em tudo: **underfitting**. Para detectar, comparamos a nota no **treino** com a nota na **validação** (dados que a rede nunca viu durante o ajuste).

Perguntas. Diagnostique cada cenário:

Cenário	Acerto no treino	Acerto na validação
A	99%	70%
B	60%	58%
C	92%	90%

1. Qual cenário mostra overfitting? Qual mostra underfitting? Qual está saudável?
2. No cenário A, cite duas coisas que ajudariam (dicas: mais dados, parar o treino mais cedo, desligar neurônios aleatórios durante o treino, rede menor).
3. No cenário B, o que você tentaria? (dica: a rede é "pequena demais" ou o treino durou pouco?)
4. Por que **nunca** devemos usar os dados de validação para ajustar os pesos?

Nível 6 – Ajuste fino e prática de verdade

Exercício 16 – Parâmetro x hiperparâmetro, e por que normalizar

Conceito. Existem dois tipos de "botões". Os **parâmetros** (pesos e bias) a **rede aprende sozinha**. Os **hiperparâmetros** (taxa de aprendizado, número de camadas, tamanho do lote, número de épocas) **você escolhe** antes de apertar o play. É a diferença entre os ingredientes que a receita transforma e as configurações do forno. Além disso, a rede treina melhor quando as entradas estão em escalas parecidas (**normalização**, lembra do 1º bimestre?).

Perguntas

1. Classifique cada item como parâmetro (P) ou hiperparâmetro (H): peso · taxa de aprendizado · bias · número de camadas · tamanho do lote · número de épocas.
2. Uma rede recebe como entrada **idade** (0 a 100) e **salário** (0 a 20.000). Por que isso atrapalha o treino se os dados não forem normalizados? (dica: qual entrada vai "gritar" mais alto?)
3. Cite uma técnica de normalização vista no 1º bimestre.
4. Quem decide os hiperparâmetros e como saber se a escolha foi boa?

Exercício 17 – Otimizadores: formas mais espertas de descer a montanha

Conceito. A descida simples dá sempre o mesmo tipo de passo. Otimizadores mais modernos são como carros com tecnologia: o **Momentum** é uma bola de boliche descendo o morro, que ganha inércia e não fica pulando em qualquer irregularidade; o **Adam** é um câmbio automático que dá um tamanho de passo diferente para cada peso, conforme o histórico dele.

Perguntas

1. Qual problema do mini-batch (passos nervosos que ficam zigue-zagueando) o Momentum ajuda a amenizar?
2. Por que dar um passo diferente para cada peso pode ser mais inteligente que o mesmo passo para todos?
3. O Adam elimina a necessidade de escolher a taxa de aprendizado? Justifique.

Exercício 18 – Consultório do treino: diagnóstico por sintomas

Conceito. Treinar rede é parecido com ser médico: o paciente (a curva de perda) apresenta sintomas e você prescreve o remédio.

Perguntas. Para cada sintoma, dê a causa mais provável e uma correção:

Sintoma	Causa provável	Correção
A perda não sai do lugar, quase reta	?	?
A perda sobe e desce loucamente, sem estabilizar	?	?
A perda vira "NaN" (números gigantes que quebram tudo)	?	?
Perda de treino cai, perda de validação sobe	?	?
As primeiras camadas não mudam nunca	?	?

Exercício 19 – Caça ao bug: lendo um laço de treino

Conceito. Na implementação prática (Keras, PyTorch, etc.), o ciclo de 4 etapas do Exercício 9 vira poucas linhas de código. Quem entende o conceito acha o erro de olhos fechados. Abaixo, pseudocódigo em português, para não depender de biblioteca:

```
# Trecho A
for epoca in range(20):
    for lote in dados_treino:
        saida = rede(lote.x)                      # passo para frente
        perda = calcular_perda(saida, lote.y)     # mede o erro
        perda.retropropagar()                     # calcula a culpa de cada peso
```

```
# Trecho B
for lote in dados_treino:
    saida = rede(lote.x)
    perda = calcular_perda(saida, lote.y)
    perda.retropropagar()
    otimizador.ajustar_pesos()
```

```
# Trecho C
for epoca in range(20):
    for lote in dados_validacao:
        saida = rede(lote.x)
        perda = calcular_perda(saida, lote.y)
        perda.retropropagar()
        otimizador.zerar_culpas()
        otimizador.ajustar_pesos()
```

Perguntas

1. No Trecho A, a perda nunca melhora, mesmo depois de 20 épocas. Qual etapa do ciclo foi esquecida?

2. No Trecho B, o código funciona, mas as culpas de lotes anteriores vão se **acumulando** e o treino fica estranho. O que falta e onde deveria entrar?
3. No Trecho C, a nota na validação fica ótima, mas a rede falha no mundo real. Qual erro conceitual foi cometido?
4. Associe cada linha do Trecho B às 4 etapas do ciclo (passo para frente, medir erro, retropropagar, ajustar).

Exercício 20 – Desafio final: juntando tudo

Conceito. Este exercício amarra todos os anteriores. Ele cobra o caso clássico em que uma só reta não resolve (o **XOR**) e um diagnóstico completo de uma rede mal configurada.

Parte 1 – XOR. A tabela abaixo é a função XOR ("ou exclusivo": dá 1 quando as entradas são diferentes).

Entrada 1	Entrada 2	Saída desejada
0	0	0
0	1	1
1	0	1
1	1	0

1. Por que um único neurônio (perceptron), que só consegue traçar **uma reta**, não consegue resolver o XOR? (dica: desenhe os 4 pontos num plano e tente separar os 1 dos 0 com uma linha.)
2. Uma rede 2-2-1 (2 entradas, 2 neurônios ocultos, 1 saída) consegue. Qual o papel da camada oculta, em linguagem de gente?
3. Narre **um passo completo de treino** dessa rede usando as 4 etapas (para frente, erro, culpa, ajuste) e dizendo, em cada etapa, o que acontece com os pesos.

Parte 2 – Diagnóstico. Um colega montou esta rede: 8 camadas ocultas com Sigmóide, todos os pesos iniciados em zero, taxa de aprendizado = 5, e entradas não normalizadas (valores de 0 a 20.000). A perda virou NaN já nas primeiras iterações.

4. Liste pelo menos **três** problemas nessa configuração e a correção de cada um, citando o exercício onde o conceito apareceu.

Parte 3 – Ensine de volta.

5. Explique a retropropagação em até 5 linhas, como se estivesse falando com sua avó. Use uma analogia própria (não vale reaproveitar as desta lista).